楽曲中の時間推移に応じた音楽ジャンル分析システムの開発

福知山公立大学 情報学部情報学科

32145037　金梨沙

指導教員　橋田光代

提出日　2025 年 1 月 31 日

目次

1 はじめに 1

2 先行研究 1

3 研究概要 1

3.1 データベース 1
3.2 データの前処理 2
3.3 音色の抽出 2
3.4 音楽ジャンルの解析 2
3.5 計算結果の表示 2

4 実装の詳細 2

4.1 使用ライブラリ 3
4.2 Calculate-genre-match-rate 関数 3
4.3 Plot-genre-histograms 関数 3
4.4 Process-existing-folder 関数 3
4.5 upload-file 関数 4
4.6 GUI の構築 4

5 検証 4

5.1 「Slow Motion Dream」での検証 4
5.2 「EMPIRE」での検証 5

6 今後の課題 6

7 おわりに 6

図目次

1 システム構成図 1
2 データベース（一部） 2
3 音声ファイル入力の画面 2
4 解析完了時の画面 3
5 Calculate-genre-match-rate 関数のソースコード 3
6 Plot-genre-histograms 関数のソースコード . . 3
7 Process-existing-folder 関数のソースコード . 3
8 upload-file 関数のソースコード 4
9 GUI のソースコード 4
10 Slow Motion Dream の jazz ヒストグラム . . 4
11 Slow Motion Dream の rock ヒストグラム . . 4
12 Slow Motion Dream の classic ヒストグラム . 5
13 0 秒から 5 秒で取得したテキスト 5
14 5 秒から 10 秒で取得したテキスト 5
15 10 秒からおわりまでで取得したテキスト . . 5
16 EMPIRE の jazz ヒストグラム 5
17 EMPIRE の rick ヒストグラム 5
18 EMPIRE の classic ヒストグラム 5

表目次

1. はじめに

近年インターネットの普及に伴い音楽の楽しみ方が大きく変化している．従来では CD を購入してプレイヤーで楽しむことが一般的であったが，近年はインターネット配信で楽しむ人が増加している．各音楽配信サイトなどでは，ユーザの嗜好に合った楽曲の検索を容易にするために楽曲を音楽ジャンル別で分類されることが多くみられる．

しかし，楽曲のジャンルは時系列で変わっていくのではないだろうか？

例として，「Mrs. GREEN APPLE」の「ライラック」?があげられる．この楽曲は，冒頭約 10 秒間でギターにエフェクトを加えた電子音が流れる．しかし，その後同じフレーズをドラムやギターの音色で構成する．これにより，電子音のような音色では感じることのなかったロックミュージックの印象を抱かせるものとなっている．このように，聴取する箇所によって抱く印象が異なる楽曲が存在すると考える．

また，音楽ジャンル解析は古くから数多くの研究が行われており，近年では主要な研究テーマの 1 つとして扱われている．しかし，これらの大半は 1 つの楽曲を 1 つの音楽ジャンルに分類するものである．このような音楽ジャンルの分類方法では，楽曲内における音楽ジャンルの変化の識別が困難であると考える．

そこで本研究では，数ある音響特徴量の中でも音色に着目し，対象楽曲の楽器構成から音楽ジャンルの変化を時系列で表示するシステムを提案することを目的とする．

2. 先行研究

現在まで，音楽ジャンル解析に関する研究は数多く行われている．楽曲のパート編成やピッチ，音長，リズムパターン，音色，リズム特性などといった特徴量を対象とした研究がある．

土橋佑亮ら?は音響信号の中でもベースパートに着目し，ベースパートの特徴量，音色，リズムを用いた音楽ジャンル推定を行っている．また，鈴木崇也ら?は旋律パートのみを用いて音楽ジャンル判別を行うための新たな特徴量を提案し，その有効性を検証している．

Basili ら [1] は MIDI データから楽曲のパート編成やピッチ，音長といった高次の特徴量を抽出し，機械学習アルゴリズムを用いて音楽ジャンルを分類している．

上記で記した研究は 1 つの楽曲に対して 1 つのジャンルを判別するものであり，従来はこの判別方法が典型であった．しかし，本研究では楽曲を時間で区切ることにより，楽曲内の音楽ジャンルの変化を可視化することが可能となる．

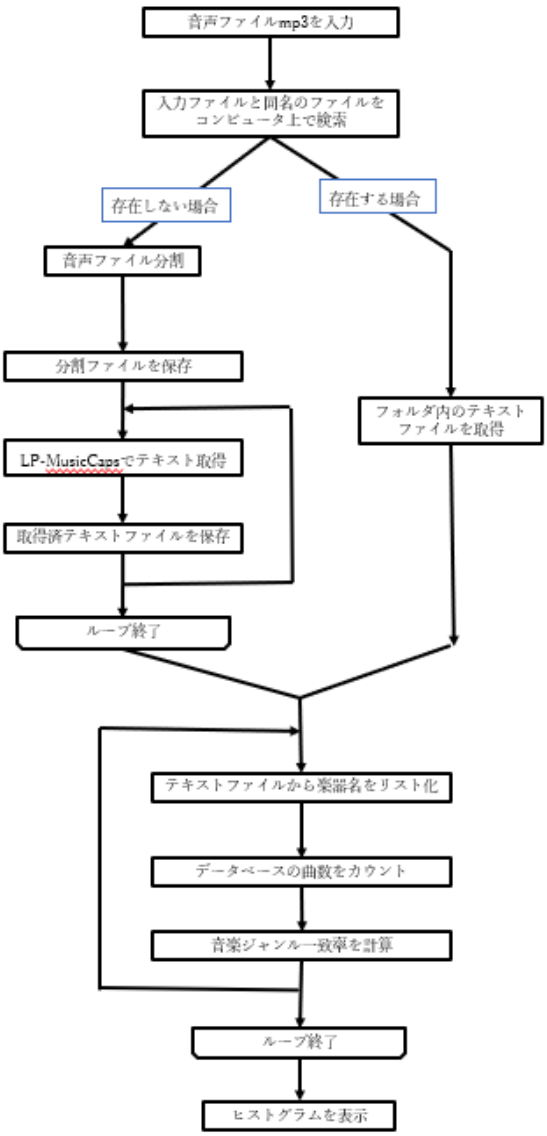

図 1: システム構成図

3. 研究概要

図 ??は，提案システムのシステム構成図である．

分析する楽曲を対象楽曲とし，対象楽曲の入力ファイルは.mp3 形式を想定する．分析結果はジャンルごとにヒストグラムで表示する．

提案システムの構成は主に音色の抽出，音楽ジャンルの解析，計算結果の表示の 3 つにわけることが出来る．データベースと入力データの前処理について記述した後，システムの詳細を記述する．

3.1 データベース

音楽ジャンル分析には，独自に収集したデータセットを使用する．jazz，classic，rock の 3 ジャンルを対象とし，各ジャンル 50 曲，合計 150 曲の楽曲を選定した．使用楽曲

© 2023-2025 Department of Informatics, The University of Fukuchiyama, Japan

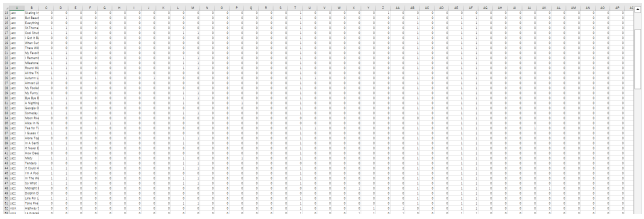

図 2: データベース（一部）

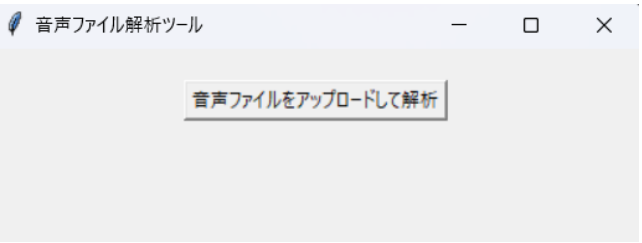

図 3: 音声ファイル入力の画面

は，音楽配信サービス Spotify にて各ジャンルの名盤を検索し，人気順で選定した．また，正確なジャンル分類を保証するため，Apple Music および Amazon Music といった他の音楽配信サービスのジャンル情報とも一致する楽曲のみを選定した．

楽曲情報は Spotify を主なデータソースとし，スコアや楽曲クレジット情報を元に使用されている楽器構成を抽出した．また，一部の楽曲についてスコアや詳細な楽曲クレジット情報が不足している場合には，聴取による推定や補足を行った．

なお，ソフトウェア音源や MIDI 音源を使用して楽器の音色を再現している楽曲については，再現している楽器として抽出した．また，楽器の音ではないシンセサイザについては，電子音とした．

作成したデータベースの一部を図 ??に示す．

3.2 データの前処理

入力された音声ファイルを 5 秒ごとのセグメントに分割する．音声ファイル名のフォルダを新たに作成し，作成フォルダ内に各セグメントを格納する．音声ファイル名のフォルダが既に存在する場合には，この処理は行わずに既存のセグメントを解析に用いる．これにより，データの重複を防ぐことが可能である．

3.3 音色の抽出

１はじめにで記述したように，本研究では時間水位における楽器構成の変化を捉え，対象楽曲の楽器構成から音楽ジャンルの変化を分析する．

音色の抽出には LP-MusicCaps: LLM-Based Pseudo Music Captioning?を使用する．これは ISMIR 2023?で発表されたもので，音楽トラックに対して自然言語によるキャプションを自動生成するものである．

表 2 は，LP-MusicCaps の整合性を示したものである．使用楽器数が少ない楽曲では高い整合性があることがわかる．しかし，楽器構成が複雑になるほど正確に抽出することが困難である傾向があると考える．

図 2 は音色の抽出手法のシステム構成図である．

LP-MusicCaps は音声データからテキスト生成するテキストである．音声ファイルを LP-MusicCaps の外部 web サイトを介して生成されたテキストを取得する．LP-MusicCaps で生成されたテキストはセグメントが保存されているフォルダ内にテキストファイルとして保存される．

しかし，外部 web サイトを介した処理を行う場合に処理速度が遅くなるという問題がある．解決方法として，すでに入力音声ファイルと同名のフォルダが存在する場合には LP-MusicCaps を介した処理を行わず，フォルダ内に存在するテキストファイルからテキストを取得する処理を追加した．これにより，過去に解析したことのある楽曲を再度解析する際の処理速度が短縮される．

取得されたテキストから正規表現を用いて楽器名のみを抽出する処理を施し，解析したセグメントの楽器構成をリスト化して取得する．

以上の処理を各セグメントに対して行い，時間ごとの楽器構成を抽出する．

3.4 音楽ジャンルの解析

本研究では，対象楽曲の楽器構成と一致する楽器構成の楽曲がデータベースに何曲存在するかを計算した数値をジャンルの一致率と定義する．

一致率はジャンルごとに計算する．ジャンルの一致率 (G) の計算式は以下のとおりである．

ジャンルの一致率 G は最大値を 1 として，数値が大きいほど一致度が高いことを示す．

対象楽曲の楽器構成を取得すると，まずデータベースの各ジャンルの総楽曲数をカウントする．次にデータベース上の楽曲から取得された楽器構成とマッチする楽曲をカウントし，上記の計算式を用いてジャンルの一致率を算出する．

上記の処理をセグメントごとに各ジャンル行う．

3.5 計算結果の表示

計算結果はヒストグラムで表示する．一致率をジャンルごとのヒストグラムで可視化し，横軸をセグメント，縦軸を一致率とする．

また，TKinter を使用して，ユーザが音声ファイルの入力を安易に行えるような GUI を作成した (図 ??，図 ??).

4. 実装の詳細

本章では，作成したシステムの試用ライブラリと主要な

© 2023-2025 Department of Informatics, The University of Fukuchiyama, Japan

図 4: 解析完了時の画面

```python
# ジャンル一致率を計算する関数
def calculate_genre_match_rate(instruments, data):
    matching_counts = data[data[instruments].eq(1).all(axis=1)].groupby('genre').size()
    genre_counts = data['genre'].value_counts()
    genre_match_rate = (matching_counts / genre_counts).fillna(0)
    return pd.DataFrame({
        'Total Count': genre_counts,
        'Matching Count': matching_counts,
        'Match Rate (ratio)': genre_match_rate
    }).fillna(0)
```

図 5: Calculate-genre-match-rate 関数のソースコード

関数についての解説を記述する．

4.1 使用ライブラリ

・os：ファイルやディレクトリの操作に使用するライブラリである．特に，フォルダ内のファイルリスト取得やパス操作に使用した．

・tkinter：GUI を構築するためのライブラリである．本システムでは，ファイルのアップロードや結果の表示に利用した．

・pandas：データフレーム形式でのデータ操作を効率化するライブラリである．本研究では csv ファイルの読み込みやデータ集計の処理に利用した．

・re：正規表現を用いた文字列操作のライブラリである．本システムでは楽器名の抽出の際に利用した．

・matplotlib：データの可視化に関するライブラリである．本システムでは，結果をヒストグラムとして表示するために使用した．

4.2 Calculate-genre-match-rate 関数

本関数は，楽器名リストとデータフレームを入力として受け取り，音楽ジャンルごとの一致率を計算する関数である．関数のソースコードを図 ??に示す．

主な処理内容は以下のとおりである．

引数は”instruments”（楽器名リスト），”data”（データベース）である．

楽器列が 1 である行を抽出，フィルタリングし，音楽ジャンルごとに一致行をカウントする．次に”genre”列全体の音楽ジャンルごとの総数をカウントし，カウントした総数と一致行数を用いて一致率を算出する．計算式は 3 章で示した通りである．

出力は，以下の 3 つのカラムを持つデータフレームを返す．

・Total Count：ジャンルの総曲数

```python
# ヒストグラムを表示する関数
def plot_genre_histograms(segment_results):
    genres = list(next(iter(segment_results.values())).index)
    plot_window = tk.Toplevel(root)
    canvas = tk.Canvas(plot_window)
    scrollbar = tk.Scrollbar(plot_window, orient="vertical", command=canvas.yview)
    scrollable_frame = tk.Frame(canvas)

    scrollable_frame.bind("<Configure>", lambda e: canvas.configure(scrollregion=canvas.bbox("all")))
    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y")

    for genre in genres:
        fig, ax = plt.subplots(figsize=(8, 6))
        segments, match_rates = zip(*[(seg, res.loc[genre, 'Match Rate (ratio)']) for seg, res in segment_results.items()])
        ax.bar(segments, match_rates)
        ax.set_xlabel("セグメント")
        ax.set_ylabel("該当率（比率）")
        ax.set_title(f"ジャンル: {genre} のセグメントごとの該当率")
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        ax.grid(axis='y')

        canvas_figure = FigureCanvasTkAgg(fig, scrollable_frame)
        canvas_figure.draw()
        canvas_figure.get_tk_widget().pack(pady=10)
```

図 6: Plot-genre-histograms 関数のソースコード

```python
# フォルダ処理関数
def process_existing_folder(folder_path, csv_file_path):
    data = pd.read_csv(csv_file_path, encoding="utf-8-sig")
    detailed_pattern = r"(?i)(electric guitar|acoustic guitar|bass guitar|bass|guitar|harp|timpani|triangle|cymbal|harpsichord|male vocal|female vocal|backing vocal|whistle|bassoo|kick|sn
    segment_results = {}

    for file in sorted(os.listdir(folder_path)):
        if file.endswith(".txt"):
            with open(os.path.join(folder_path, file), "r", encoding="utf-8") as f:
                result = f.read()

            print(f"\n{file} の内容:\n{result}")

            # 指定されたコード部分を保持
            matches = re.findall(detailed_pattern, result)
            matches = [
                "drum" if item.lower() in {"kick", "snare", "drums", "drum", "drumming", "hi-hat"} else
                "vocal" if item.lower() in {"male vocal", "female vocal", "vocals", "vocalist", "voice"} else
                "bass" if item.lower() in {"bass", "bass guitar"} else
                "electric guitar" if item.lower() in {"e-guitar"} else
                item.lower()
                for item in matches
            ]
            matches = list(set(matches))
            print(f"{file} で検出された楽器名: {matches}")

            if matches:
                genre_result = calculate_genre_match_rate(matches, data)
                segment_results[file] = genre_result

    if segment_results:
        plot_genre_histograms(segment_results)
```

図 7: Process-existing-folder 関数のソースコード

・Matching Count：楽器構成が一致している曲数

・Match Rate(ratio)：音楽ジャンルの一致率

4.3 Plot-genre-histograms 関数

本関数は，音楽ジャンルごとのセグメントごとの一致率をヒストグラムで表示する関数である．

関数のソースコードを図 ??に示す．

ヒストグラムの表示は tkinter ライブラリを使用してサブウィンドウを表示して行う．引数は”segment-results”（各セグメントの音楽ジャンル該当率）である．

各セグメントにおける一致率を取得後，縦軸を音楽ジャンル一致率の最大値，横軸をセグメント（時間経過）としたヒストグラムをサブウィンドウに表示する．サブウィンドウにはグラフ表示領域と垂直スクロールバーを作成し，フレーム内の要素が変化したときキャンバスのスクロール領域を自動調整する処理を追加している．

4.4 Process-existing-folder 関数

本関数は，フォルダ内のテキストファイルを処理し，楽器名を抽出する関数である．

関数のソースコードを図 ??に示す．

引数は”folder-path”（処理対象のフォルダパス），”csv-file-path”（csv ファイルのパス）である．

本システムでは，楽器名を正規表現パターンと定義して使用楽器を抽出している．取得した各テキストファイルの内容を小文字に変換後，正規表現で楽器名を抽出する．その際，一部の楽器名は一般的なカテゴリに変換する処理を追加した．

© 2023-2025 Department of Informatics, The University of Fukuchiyama, Japan

```python
# ファイルアップロード関数
def upload_file():
    file_path = filedialog.askopenfilename(
        title="音声ファイルを選択してください",
        filetypes=[("音声ファイル", "*.mp3 *.wav *.ogg *.flac"), ("すべてのファイル", "*.*")]
    )
    if file_path:
        folder_path = os.path.splitext(file_path)[0]
        csv_file_path = "testdata.csv"
        if os.path.exists(folder_path):
            process_existing_folder(folder_path, csv_file_path)
            messagebox.showinfo("処理完了", f"{folder_path} 内のファイルを処理しました。")
```

図 8: upload-file 関数のソースコード

```python
# メインウィンドウ
root = tk.Tk()
root.title("音声ファイル解析ツール")
tk.Button(root, text="音声ファイルをアップロードして解析", command=upload_file).pack(pady=20)
root.mainloop()
```

図 9: GUI のソースコード

次に，データベースと一致する楽器がある場合，”calculate-genre-match-rate” 関数を用いて音楽ジャンルの一致率を計算する．

計算結果は” segment-results” 辞書に保存され、” plot-genre-histograms” 関数を用いて音楽ジャンルごとの一致率をヒストグラムで表示する．この際，セグメントの音楽ジャンル一致率の計算結果を引数に取る．

4.5 upload-file 関数

本関数は，ユーザが音声ファイルをアップロードするための関数である．関数のソースコードを図 ??に示す．

本システムの入力は mp3 であるため，選択可能なファイル形式は.mp3 のみに指定している．また、データベースは固定のパスとして設定している．

指定パスが存在し，ディレクトリであるかを確認後，指定フォルダ内のテキストファイルを処理する．処理内容は process-existing-folder 関数の内容である．

フォルダが存在しない場合，つまり解析したことのないファイルを解析する場合には以下の処理に分岐する．Split-audio-to-named-dir 関数を用いて音声ファイルを 5 秒ごとのセグメントに分割する．分割されたセグメントを，web サイトを介して処理する．

4.6 GUI の構築

ソースコードを図 ??に示す．

GUI は tkinter を用いて作成する．

ウィンドウのタイトルバーに表示される文字列を「音声ファイル解析ツール」に設定する．プログラムを実行すると，音声ファイルのアップロードボタンがウィンドウに表示される．

その他の GUI は各関数で処理内容を記述している．

5. 検証

本章では，構築したシステムの検証手順とその結果を記述する．

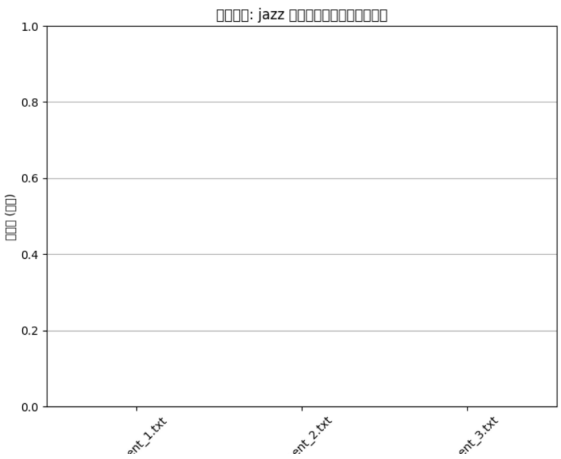

図 10: Slow Motion Dream の jazz ヒストグラム

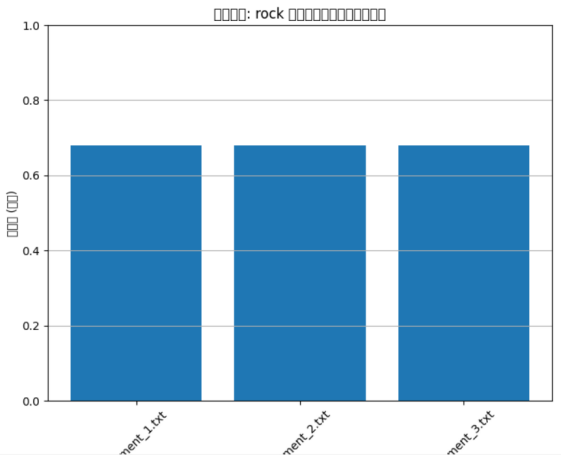

図 11: Slow Motion Dream の rock ヒストグラム

5.1 「Slow Motion Dream」での検証

入力する楽曲は，Steven M Bryant の「Slow Motion Dream」の一部を使用する．この楽曲は，インターネット上で公開されているフリーミュージックである．この音声ファイルは序盤は electric guitar，vocal で構成され，途中で drum が加わる楽器構成である．音声ファイルの時間は約 10 秒である．

検証結果のヒストグラムを図 ??，図 ??，図 ??に示す．

図 ??，図 ??，図 ??は，各セグメントで取得されたテキストファイルの内容である．

また，このテキストから取得した楽器構成は次のとおりである．

・0 秒から 5 秒：[’vocal’, ’electric guitar’, ’bass guitar’]

・5 秒から 10 秒：[’vocal’, ’electric guitar’, ’drum’, ’bass guitar’]

・10 秒からおわり：[’vocal’, ’electric guitar’]

これらの楽器構成はすべて聴取した際の楽器構成と一致しており，楽器構成や一致率の計算結果，ヒストグラムの結果は正しいといえる．

© 2023-2025 Department of Informatics, The University of Fukuchiyama, Japan

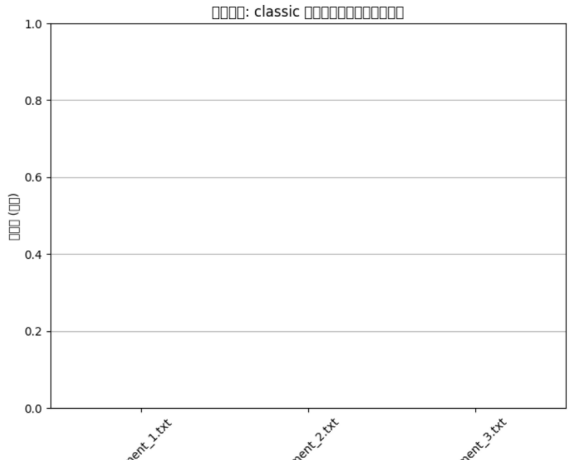

図 **12:** Slow Motion Dream の classic ヒストグラム

```
[0:00-10:00]
This is a pop/soft rock music piece. There is a male
vocalist singing melodically in the lead. The main melody
is being played by the electric guitar and the keyboard
while the bass guitar is playing in the background. The
rhythm is provided by a basic acoustic drum beat. The
atmosphere is groovy. This piece could be used in the
soundtrack of a feelgood movie, especially during the
scenes where a character is trying to break free.
```

図 **13:** 0 秒から 5 秒で取得したテキスト

```
[0:00-10:00]
The low quality recording features a rock song that
consists of a passionate female vocal, alongside
harmonizing background female vocals, singing over punchy
kick and snare hits, shimmering hi hats, groovy bass guitar
and wide electric guitar melody. It sounds energetic and
addictive - like something you would listen to.
```

図 **14:** 5 秒から 10 秒で取得したテキスト

```
[0:00-10:00]
The low quality recording features a short snippet of flat
male vocal talking, after which there is a funky electric
guitar chord being played. The recording is noisy and in
mono.
```

図 **15:** 10 秒からおわりまでで取得したテキスト

5.2 「EMPIRE」での検証

入力する楽曲は snowman の「EMPIRE」冒頭 1 分 19 秒を使用する．

この楽曲はモーツァルト作曲「交響曲第 25 番ト短調 K.183」の第一楽章をサンプリングした楽曲であり，イントロと呼ばれる冒頭部分ではサンプリング元楽曲の冒頭部分である第一楽章の第一主題が流れ，その後ロック調のメロディで構成される．また，間奏部分においてもサンプリング元楽曲が流れており，classic と jazz の要素が時間水位に伴い混在する楽曲となっている．このように本楽曲は時間遷移に伴いジャンルが変化しているため，システムの検証に適していると考えた．

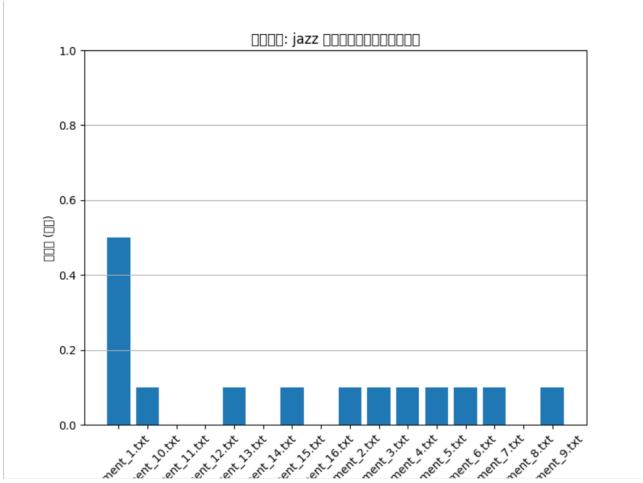

図 **16:** EMPIRE の jazz ヒストグラム

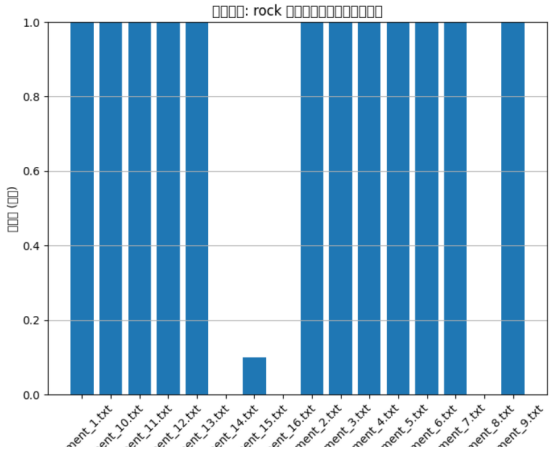

図 **17:** EMPIRE の rick ヒストグラム

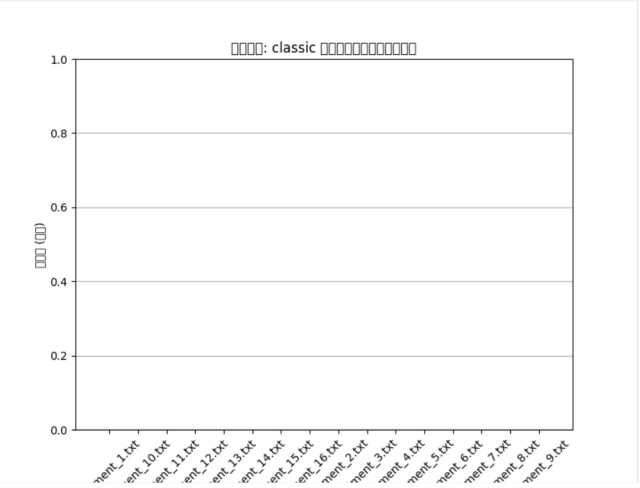

図 **18:** EMPIRE の classic ヒストグラム

検証結果のヒストグラムを図 **??**，図 **??**，図 **??**に示す．

対象楽曲の冒頭 5 秒間の楽器構成は violin，viola，cello，contrabass，oboe，faggot，corn，horn，vocal，drum である．そのため，classic の一致度が高くなることが考えられた．

しかし，実装結果は classic の一致度が 0 となっている．

データベースの Classic 楽曲には vocal が使用されている楽曲がないため，このような結果が出ていると考えられる．

また，冒頭 5 秒間のセグメントにて取得した楽器構成リストは，['vocal'] のみである．対象楽曲で試用されている楽器数が多いため，テキスト生成の際に正確に楽器を抽出できなかったものと考えられる．

6. 今後の課題

本システムでは音色の抽出に LP-MusicCaps を使用したが，楽器数が多いと正確に使用楽器が抽出できないという問題点があった．そのため，楽器数の少ない楽曲を対象楽曲とした際は正しい結果が得られたが，音数が多く複雑な楽曲を対象楽曲とした際に正しい実装結果が得られなかった．

今後の課題として，音色の抽出方法の見直しがあげられる．

また，今回は 50 曲× 3 の 150 曲のデータベースを使用した．しかし，より正確かつ汎用性の高いシステムになるために，データベースの拡大が必要であると考える．

7. おわりに

本研究では，楽曲中の時間推移に伴う音楽ジャンルの変化を分析・可視化するシステムを提案し，検証を行った．従来の 1 つの楽曲を 1 つの音楽ジャンルに分類する解析手法では捉えきれなかったジャンルの多様性に注目し，楽器構成の変化を時間軸に沿って解析することで，楽曲内でジャンルがどのように移り変わるかを可視化する新しいアプローチを示した．

検証の結果，楽器構成がシンプルな楽曲では有効な結果を得られたが，楽器構成が複雑な楽曲では音色抽出の精度に課題があることが明らかになった．例えば、特定の楽器が正確に認識されないず，データベースとの一致率に影響を及ぼすケースが確認された．また，今回使用したデータベースは 150 曲と規模が限られており，データの多様性においても改善の余地があると考える．

今後の課題として，音色抽出アルゴリズムの改良や，より大規模で多様性に富んだデータベースの構築が挙げられる．これにより，複雑な楽曲への対応力を向上させるとともに，システムの汎用性と精度を高めることが可能となる．

本研究は，音楽ジャンル解析における新たな視点を提供し，音楽の多様性を捉える手法の一端を示した．本システムが音楽理解の深化や楽しみ方の多様化に寄与することを期待する．

謝辞　書くならお好きに

参考文献

[1] Basili 0001, R., Serafini, A. and Stellato, A.: Classification of Musical Genre: A Machine Learning Approach., *Proceedings of the 5th International Conference on Music Information Retrieval*, (online), DOI: 10.5281/zenodo.1416754 (2004).

© 2023-2025 Department of Informatics, The University of Fukuchiyama, Japan